Document number: D1792R0
Date: 2019-06-16
Project: Programming Language C++
Audience: SG1
Reply-to: Christopher Kohlhoff <chris@kohlhoff.com>

Simplifying and generalising Sender/Receiver for asynchronous operations

Introduction

P1341, Unifying Asynchronous APIs in C++ Standard Library presents an API design approach where Sender/Receivers are used to, notionally, unify sources of asynchronicity. Unfortunately, this approach represents a significant backward step in flexibility, usability, and efficiency when compared to existing practice. This is primarily a consequence of P1341's design being tightly coupled to a lazy evaluation model.

This paper introduces a simpler, more general foundation for expressing the relationship between sender and receiver. This foundation enables efficient support for not just eager and lazy concurrency, but also for models that lie somewhere in-between, such as fibers.

Problems with an exclusively lazy approach

An exclusively lazy approach to asynchronicity has the following issues:

- Inefficiency: It requires that arguments to an asynchronous operation be copied (possibly in some repackaged form) so that they are still available when lazy evaluation triggers initiation of the underlying operation.

- Reduced safety: Composition techniques such as coroutines and fibers provide the tools to safely manage object lifetime in asynchronous contexts, but an exclusively lazy approach requires greater programmer attention and discipline, even when using these techniques.

Other problems with the P1341 chosen design

- Lack of flexibility: The lazy design used in P1341 makes it difficult to generically customise the behaviour of senders in the context of particular composition techniques. For example, we may wish to construct algorithms as senders that are generic across fibers and other similar synchronous contexts.

- Forced conflation of unrelated concepts: P1341 appears to conflate two disjoint concepts (Awaitable and Sender) in order to create a new concept, Task, that can support both coroutines and lazy chaining of Sender/Receiver objects. A better approach would be to adopt a more fundamental representation that can support both compositional techniques (and indeed others).

A better way

The CompletionToken mechanism was first described in N3747, A Universal Model for Asynchronous Operations, and its revisions N3964 and N4045 introduced a way to decouple asynchronous operations from the mechanisms used to compose them. This technique garnered in widespread, positive field experience in

Boost.Asio (starting with the Asio 1.10.0 / Boost 1.54 release) which later formed the basis of the Networking TS (N4771, working draft).

However, one disadvantage of the CompletionToken mechanism at that time was that it required eager initiation of the underlying asynchronous operations. This shortcoming was corrected in Asio 1.14.0 / Boost 1.70 with a small enhancement to the design to also allow lazy initiation.

A key realisation during executor discussion was that the CompletionToken mechanism could represent the same foundational concepts as the Sender/Receiver model. By combining and evolving these approaches, we enable a generalised Sender/Receiver model for asynchronous operations that supports eager, lazy, as well as hybrid models of concurrency such as fibers.

Key design elements of a generalised Sender/Receiver model

- *Sender*: A function (or function object) that takes a Receiver as one of its arguments.
- *Receiver*: An object that determines how and where the results of a Sender will be received.
- *Signals* and *Signatures*: Communication between Sender and Receiver is unidirectional and consists of a set of Signals, where each Signal has a Signature. The set of possible Signals for a particular Sender/Receiver pairing is expressed as a variadic parameter pack of Signatures.
- *`receiver_traits`*`<>`: A traits type used to determine concretely how a Receiver handles a given set of Signals, specified as Signatures.
- *Handler*: A function object, produced by a Receiver, to concretely handle the set of signals generated by a Sender. It is callable with each of the given Signatures. By default, a Handler is also a Receiver, and when used as one causes the operation to be eagerly initiated.
- *SenderLauncher*: - An object used to launch the underlying asynchronous operation associated with a Sender.
- *receiver_traits<>::connect()*: Called by a Sender to establish its relationship with a Receiver. Passed the SenderLauncher, the Receiver, and an optional list of arguments to be passed back to the SenderLauncher. The Receiver is responsible for calling the SenderLauncher with its Handler and the optional arguments. The Receiver gets to choose when the SenderLauncher is called, which allows the model to support eager, lazy, and other hybrid models of execution.
- *`connect`*`<>()`: A helper function used by a Sender to simplify the call to `receiver_traits<>::connect()`.

Example: An asynchronous operation as a Sender

As a simple example, let's implement an asynchronous operation that performs `std::accumulate` in a background thread. This operation is presented to the user as a sender:

```
template <
    class InputIt,
    class T,
    Receiver<void(T)> R              // #1
  >
auto async_accumulate(
    InputIt first,
    InputIt last,
    T init,
    R&& receiver
  )
```

```
{
  return connect<void(T)>(            // #2
      [](                             // #3
          Handler<void(T)> h,         // #4
          InputIt first,
          InputIt last,
          T init
        )
      {
        std::thread(
            [h = std::move(h), first, last, init]() mutable
            {
              h(std::accumulate(first, last, init));
            }
          ).detach();
      },
      std::forward<R>(receiver),
      first,                          // #5
      last,
      init
    );
}
```

The key design elements of the simplified Sender/Receiver model are used in this example as follows:

1. The Sender's API boundary specifies the set of signals that will be sent to the receiver.

2. The Sender establishes the connection to the Receiver, and in doing so communicates the set of signals that will be generated. The Receiver may use this opportunity to customise the function return type and value, as well as choose when the underlying asynchronous operation is launched.

3. The SenderLauncher object is passed to `connect` to encapsulate how the underlying asynchronous operation is actually launched.

4. The SenderLauncher object receives a concrete Handler that will process the signals generated by the operation.

5. Additional arguments to the SenderLauncher are passed separately, rather than being captured in the SenderLauncher lambda. This allows the Receiver to customise how these arguments will be forwarded to the SenderLauncher. For example, in eager evaluation the arguments are simply passed straight through. In lazy evaluation, they need to be captured and stored.

This function can now be used eagerly by simply specifying a Handler as the Receiver:

```
async_accumulate(
    vec.begin(), vec.end(), 0,
    [](int result)
    {
      /*...*/
    }
  );
```

However, the framework supports an infinitely extensible set of receiver types enabling operations to be used with many different compositional techniques, such as futures:

```
future<int> f = async_accumulate(vec.begin(), vec.end(), 0, use_future);

/* ... */

int result = f.get();
```

coroutines:

```
int result = co_await async_accumulate(vec.begin(), vec.end(), 0, use_await);
```

fibers (or stackful coroutines):

```
int result = async_accumulate(vec.begin(), vec.end(), 0, use_fiber);
```

all in addition to the lazy evaluation model described in P1341:

```
auto lazy_eval = async_accumulate(vec.begin(), vec.end(), 0, lazy);

/* ... */

future<int> f = lazy_eval(use_future);
// or any other type of receiver, as desired
```

Furthermore, we are able implement algorithms that are generic across categories of Receiver that behave a certain way, such as so-called synchronous Receivers that represent fiber-like and thread-like behaviour that blocks the calling execution agent. Once a vocabulary of well-known Receiver categories is established, algorithms may be overloaded to allow for further, tailored optimisations.

Appendix: Possible implementation of a simplified, generalised Sender/Receiver

```
//-----------------------------------------------------------------------------

namespace detail {

template <class F, class... Args>
concept bool Invocable = requires(F&& f, Args&&... args)
{
  std::invoke(std::forward<F>(f), std::forward<Args>(args)...);
};

template <class F, class Sig>
constexpr bool invocable_as_v = false;

template <class F, class R, class... Args>
constexpr bool invocable_as_v<F, R(Args...)> = Invocable<F, Args...>;

template <class T>
inline constexpr bool is_signature_v = false;

template <class R, class... Args>
inline constexpr bool is_signature_v<R(Args...)>  = true;

template <class T, class... Sigs>
inline constexpr bool is_handler_v = (... && invocable_as_v<T, Sigs>);

template <class T, class... Sigs>
struct receiver_traits_base;

template <class T, class... Sigs> requires is_handler_v<T, Sigs...>
struct receiver_traits_base<T, Sigs...>
{
  using receiver_type = T;

  template <class SenderLauncher, class Receiver, class... Args>
  static void connect(SenderLauncher&& s, Receiver&& r, Args&&... args)
  {
```

```
    std::invoke(
        std::forward<SenderLauncher>(s),
        std::forward<Receiver>(r),
        std::forward<Args>(args)...);
  }
};

} // namespace detail

//-----------------------------------------------------------------------------

template <class T>
concept bool Signature = detail::is_signature_v<T>;

//-----------------------------------------------------------------------------

template <class T, Signature... Sigs>
concept bool Handler = detail::is_handler_v<T, Sigs...>;

//-----------------------------------------------------------------------------

template <class T, Signature... Sigs>
struct receiver_traits : detail::receiver_traits_base<T, Sigs...> {};

//-----------------------------------------------------------------------------

template <class T, Signature... Sigs>
concept bool Receiver = requires(T&&)
{
  typename receiver_traits<std::decay_t<T>, Sigs...>::receiver_type;
};

//-----------------------------------------------------------------------------

template <
    Signature... Sigs,
    class SenderLauncher,
    Receiver<Sigs...> R,
    class... Args
  >
auto connect(SenderLauncher&& s, R&& r, Args&&... args)
{
  return receiver_traits<std::decay_t<R>, Sigs...>::connect(
      std::forward<SenderLauncher>(s),
      std::forward<R>(r),
      std::forward<Args>(args)...);
}

//-----------------------------------------------------------------------------
```